Laboratório Prolog · Aula 13

Fernando Queiroz · RA 24038237-2 · Turma B
Gerado em 20/10/2026, 20:27:48 · tempo registrado no cronômetro: 36:58

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) → L = scheme.
paradigma(java, logico) → false.

Saída do ola.pl

```
1 -

?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.

2 -

?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(java,A).
   paradigma(java,orientado_a_objetos)
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) pergunta qual é o paradigma do Prolog, retornando P = logico. Já paradigma(prolog, logico) verifica se o fato de que Prolog pertence ao paradigma lógico está registrado no programa, retornando verdadeiro ou falso.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Significa que o programa não possui um fato ou uma regra que associe Java ao paradigma lógico. Não significa que Java não possa utilizar conceitos de programação lógica, significa apenas que essa associação não está registrada

no programa.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

A consulta mulher(Quem) terá 7 respostas, pois existem 7 fatos cadastrados para mulher/1.

As respostas serão apresentadas nesta ordem:

1. Quem = lucia
2. Quem = marta
3. Quem = tereza
4. Quem = joana
5. Quem = bia
6. Quem = ana
7. Quem = dora

Já a consulta (mae(M, F), mulher(F)) terá 4 respostas, pois busca os fatos em que F é filha de uma mãe e também está cadastrada como mulher.

Os pares serão:

1. (M = lucia, F = tereza)
2. (M = marta, F = joana)
3. (M = tereza, F = bia)
4. (M = joana, F = ana)

Saída do familia.pl

```
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
```

```
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

A consulta pergunta quais são as mães (Mae) cujos filhos (Filha) são mulheres. Teve 4 respostas: (marta, joana), (tereza, bia), (joana, ana) e (lucia, tereza).

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

O pressuposto é o mundo fechado: se algo não pode ser provado pela base de conhecimento, é considerado falso. Em situações reais, isso pode gerar conclusões incorretas, pois a informação pode simplesmente não estar cadastrada.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

As respostas apareceram na ordem: lucia, marta, tereza, joana, bia, ana, dora. A ordem é determinada pela sequência dos fatos mulher/1 na base, que o Prolog percorre de cima para baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake): X = paulo.
primo(leo, X): X = jake e X = ana

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

f(X, X) = f(a, b): falha.
f(X, X) = f(a, Y): unifica, com X = a e Y = a.
[A|B] = []: falha.
2 + 3 = 5: falha, pois unificação não calcula expressões aritméticas.

Saídas de regras.pl e unificacao.pl

```
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
   primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
```

```
    ancestral(osvaldo,vern)
    ancestral(osvaldo,tereza)
    ancestral(osvaldo,joana)
    ancestral(osvaldo,paulo)
    ancestral(osvaldo,jake)
    ancestral(osvaldo,ana)
    ancestral(osvaldo,leo)
    ancestral(osvaldo,bia)
 ?- ancestral(A,jake).
    ancestral(joana,jake)
    ancestral(jorge,jake)
    ancestral(lucia,jake)
    ancestral(marta,jake)
    ancestral(osvaldo,jake)
    ancestral(vern,jake)
 ?- ancestral(jake,A).
    false.
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

A regra precisa de duas cláusulas: o caso base verifica se X é progenitor direto de Y, o caso recursivo verifica se X é progenitor de alguém que também é ancestral de Y

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, uma pessoa pode ser considerada irmã de si mesma. Essa condição garante que sejam pessoas diferentes

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

A primeira não unifica porque X teria que ser a e b ao mesmo tempo. A segunda unifica com X = a e Y = a

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Sim, mas seria necessário implementar a lógica nas funções. No Prolog, a mesma regra permite consultar a relação em diferentes direções, graças à unificação

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

concatena(X, Y, [1, 2, 3]) terá 4 respostas, dividindo a lista em todas as posições possíveis:
- X = [], Y = [1, 2, 3]
- X = [1], Y = [2, 3]
- X = [1, 2], Y = [3]
- X = [1, 2, 3], Y = []

Saída do listas.pl

```
 --- membro: serve para testar E para gerar
 ?- membro(b,[a,b,c]).
    membro(b,[a,b,c])
 ?- membro(z,[a,b,c]).
    false.
 ?- membro(A,[a,b,c]).
    membro(a,[a,b,c])
    membro(b,[a,b,c])
    membro(c,[a,b,c])
 --- tamanho
```

```
?- tamanho([a,b,c],A).
   tamanho([a,b,c],3)
?- tamanho([],A).
   tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
   concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
   concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
   inverte([1,2,3],[3,2,1])
Ready
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Caso base: lista vazia, com tamanho zero. Passo recursivo: contar os elementos da cauda e somar 1. A cada chamada, fica pendente somar 1 ao tamanho retornado.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Há quatro respostas porque a lista pode ser dividida em quatro posições. O anexar do Scheme apenas calcula a lista resultante; não permite descobrir diretamente as listas originais a partir do resultado.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O mecanismo de unificação e backtracking permite encontrar cada elemento como uma resposta diferente, sem precisar de um laço explícito.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

não respondido

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- A=2+3.
   2+3=2+3
?- A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- A is 7 mod 3.
   1 is 7 mod 3
?- A is 7//2.
   3 is 7//2
?- A is 2^10.
```

```
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
   fatorial(5,120)
?- fatorial(0,A).
   fatorial(0,1)
?- fib(10,A).
   fib(10,55)
?- soma([1,2,3,4],A).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
   ERRO: instantiation_error
?- A is B+1.
   ERRO: instantiation_error
Ready
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= unifica termos (X = a); is calcula expressões (X is 2 + 3); =:= compara resultados numéricos (5 =:= 2 + 3).

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

fatorial(N, 120) dá erro porque a regra faz operações aritméticas com N ainda desconhecido. Já concatena funciona em várias direções porque usa unificação e recursão, sem exigir cálculos numéricos com variáveis livres.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Ambos usam recursão e um caso base. Em Haskell, o resultado é retornado pela função; em Prolog, o resultado é associado a um argumento extra, como fatorial(5, F)

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro1(X): encontra homens que não estão cadastrados como casados.

solteiro2(X): dá false, pois tenta verificar casado(X) antes de X ter valor.

maximo2(5, 3, M): tem duas respostas, M = 5 e M = 3.

maximo3(5, 3, 3): dá true, incorretamente, porque o corte impede tentar a segunda cláusula, mas não desfaz a escolha de M = 5 da primeira cláusula.

Saída de controle.pl e mensagem de ancestral_ruim

```
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
```

```
    tenta ana e bia
    tenta ana e caio
    resposta: mesmo_gosto(ana,caio)
    tenta bia e ana
    tenta bia e bia
    tenta bia e caio
    tenta caio e ana
    resposta: mesmo_gosto(caio,ana)
    tenta caio e bia
    tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- solteiro2(A).
   false.
--- C) corte
?- maximo1(5,3,A).
   maximo1(5,3,5)
?- maximo2(5,3,A).
   maximo2(5,3,5)
   maximo2(5,3,3)
?- maximo3(5,3,A).
   maximo3(5,3,5)
?- maximo1(5,3,3).
   false.
?- maximo3(5,3,3).
   maximo3(5,3,3)
Ready

ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)

Fatal Error: local stack overflow (size: 16384 Kb, reached: 16383 Kb, environment variable
used: LOCALSZ)
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

3 cores: 24 colorações.

2 cores: 0 colorações.

4 cores: vou conferir a contagem exata com as restrições do código antes de afirmar o resultado.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
```

```
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
   false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog encontra (ana, caio) e (caio, ana), pois ambos gostam de café. Em tenta ana e ana, falha em X \== Y e volta para buscar outro valor para Y

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

solteiro2(X) retorna false porque tenta verificar se X não é casado antes de atribuir um valor a X. solteiro1(X) funciona porque primeiro encontra os homens. Osvaldo aparece como solteiro porque ser pai não significa estar cadastrado como casado

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte não protegeu porque, na primeira cláusula, a unificação do terceiro argumento ocorre antes da execução do corte. Corrigindo:

maximo3(X, Y, R) :- X >= Y, !, R = X.
maximo3(_, Y, Y).

Assim, maximo3(5, 3, 3) retorna false.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim estoura a pilha porque chama a si mesma antes de verificar um progenitor, entrando em recursão infinita. ancestral_bom verifica primeiro o progenitor e depois faz a chamada recursiva.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

- 3 cores: 24 colorações.
- 2 cores: 0 colorações.
- 4 cores: 1.728 colorações.

O programa é declarativo porque descreve as condições de uma coloração válida, deixando o Prolog encontrar as soluções por backtracking.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No paradigma lógico, como no Prolog com concatena, descrevemos relações entre dados e deixamos o sistema buscar as soluções possíveis. Já no funcional, como no Scheme com anexar, utilizamos funções que recebem valores e retornam um resultado.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

Uma deficiência do Prolog é a dependência da ordem das regras, como observado em ancestral_ruim, que pode causar buscas ineficientes ou infinitas. Outra é a dificuldade de lidar com cálculos aritméticos de forma relacional, como em fatorial(N, 120), pois o uso de is exige que a expressão do lado direito esteja calculável.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas que envolvem regras lógicas, relacionamentos e busca de soluções, como sistemas especialistas e consultas a bases de conhecimento. Não escolheria para sistemas comerciais complexos, como um ERP, pois Java oferece maior adequação à construção de aplicações com regras de negócio, persistência de dados, interfaces e integração com outros sistemas.